

Algorithm Complexity

Tutorial 1

Exercise 1

Consider a modern computer capable of executing approximately 10^{15} basic instructions per second. An algorithm operating on an input of size n performs a number of instructions given by one of the following functions:

$$f(n) \in \{\log_2 n, n, n \log_2 n, n^2, n^3, 2^n\}.$$

- For each function $f(n)$ and each input size $n \in \{6, 25, 50, 100\}$, calculate the number of instructions performed by the algorithm and the corresponding execution time.
- Express the execution times using an appropriate time unit and complete a table summarizing the results.
- For each input size, arrange the algorithms from the fastest to the slowest according to their execution time.
- Arrange the six functions according to their asymptotic growth rate, from the slowest-growing to the fastest-growing function.
- Consider an algorithm with complexity 2^n and $n = 100$. Suppose that the computer used in the previous questions is replaced by a computer that is 10 000 times faster. Calculate the new execution time and compare it with the original execution time.
- Moore's law is commonly expressed as an approximate doubling of computing capacity every two years. Suppose that a computer in 2005 could execute 40×10^{12} instructions per second.
 - Estimate the computing capacity of this computer in 2025, assuming that the computing capacity doubles every two years.
 - Using the estimated 2025 computing capacity, calculate the execution time of an algorithm performing 2^{100} instructions.
 - Express the resulting execution time in years and discuss whether the improvement predicted by Moore's law makes the computation practically feasible.
- Based on the previous results, explain why improvements in computing capacity cannot compensate indefinitely for algorithms with rapidly growing complexity.

Additional Question

Consider two algorithms solving the same problem:

$$f_A(n) = n^4 \quad \text{and} \quad f_B(n) = 2^n.$$

For small input sizes, their execution times may appear comparable. Explain what happens as n increases and discuss which algorithm is more suitable for large-scale inputs.

Solution

The computer can execute approximately

$$10^{15}$$

basic instructions per second. Therefore, the execution time for an algorithm performing $f(n)$ instructions is

$$T(n) = \frac{f(n)}{10^{15}} \text{ seconds.}$$

1. Number of instructions and execution time

For the input sizes $n = 6, 25, 50, 100$, we obtain the following results.

$f(n)$	$n = 6$	$n = 25$	$n = 50$	$n = 100$
$\log_2 n$	2.585	4.644	5.644	6.644
n	6	25	50	100
$n \log_2 n$	15.510	116.096	282.193	664.386
n^2	36	625	2500	10000
n^3	216	15625	125000	1000000
2^n	64	33,554,432	1.1259×10^{15}	1.2677×10^{30}

Since the computer performs 10^{15} instructions per second,

$$T(n) = \frac{f(n)}{10^{15}}.$$

For example, for $f(n) = n^3$ and $n = 100$:

$$T(100) = \frac{100^3}{10^{15}} = \frac{10^6}{10^{15}} = 10^{-9} \text{ seconds.}$$

For $f(n) = 2^n$ and $n = 100$:

$$T(100) = \frac{2^{100}}{10^{15}} \approx 1.2677 \times 10^{15} \text{ seconds.}$$

Converting this value into years:

$$T(100) = \frac{1.2677 \times 10^{15}}{365.25 \times 24 \times 3600} \approx 4.02 \times 10^7 \text{ years.}$$

Thus, the execution times are:

$f(n)$	$n = 6$	$n = 25$	$n = 50$	$n = 100$
$\log_2 n$	$2.58 \times 10^{-15} s$	$4.64 \times 10^{-15} s$	$5.64 \times 10^{-15} s$	$6.64 \times 10^{-15} s$
n	$6 \times 10^{-15} s$	$2.5 \times 10^{-14} s$	$5 \times 10^{-14} s$	$10^{-13} s$
$n \log_2 n$	$1.55 \times 10^{-14} s$	$1.16 \times 10^{-13} s$	$2.82 \times 10^{-13} s$	$6.64 \times 10^{-13} s$
n^2	$3.6 \times 10^{-14} s$	$6.25 \times 10^{-13} s$	$2.5 \times 10^{-12} s$	$10^{-11} s$
n^3	$2.16 \times 10^{-13} s$	$1.5625 \times 10^{-11} s$	$1.25 \times 10^{-10} s$	$10^{-9} s$
2^n	$6.4 \times 10^{-14} s$	$3.355 \times 10^{-8} s$	$1.126 s$	$4.02 \times 10^7 \text{ years}$

2. Ranking the algorithms

For every sufficiently large n , the functions can be ordered according to their growth rates as follows:

$$\log_2 n < n < n \log_2 n < n^2 < n^3 < 2^n$$

Therefore, for large input sizes, the corresponding algorithms are expected to execute from fastest to slowest in this same order.

An interesting observation is that for $n = 6$, the difference between some algorithms is relatively small. However, as n increases, the differences become much more significant.

For example, for $n = 100$:

$$n^3 = 10^6,$$

whereas

$$2^{100} \approx 1.27 \times 10^{30}.$$

Thus, the exponential function grows enormously faster than the polynomial functions.

3. Increasing the computing capacity by 10,000 times

The original computer performs

$$10^{15}$$

instructions per second.

A computer that is 1,000 times faster performs

$$10^{15} \times 10^4 = 10^{19}$$

instructions per second.

For $f(n) = 2^n$ and $n = 100$:

$$T(100) = \frac{2^{100}}{10^{19}}.$$

Since

$$2^{100} \approx 1.2677 \times 10^{30},$$

we obtain

$$T(100) \approx 1.2677 \times 10^{11} \text{ seconds.}$$

Converting to years:

$$T(100) \approx \frac{1.2677 \times 10^{11}}{31,557,600} \approx 4.02 \times 10^3 \text{ years.}$$

Therefore,

$$T(100) \approx 4,020 \text{ years}$$

even with a computer that is 10,000 times faster.

This shows that a fixed improvement in hardware performance does not make an exponential computation practically feasible.

4. Moore's Law

According to the assumption in the exercise, computing capacity doubles every two years.

The initial computing capacity in 2005 is

$$40 \times 10^{12}$$

instructions per second.

The period from 2005 to 2025 is

$$2025 - 2005 = 20 \text{ years.}$$

Therefore, the number of doubling periods is

$$\frac{20}{2} = 10.$$

The improvement factor is consequently

$$2^{10} = 1024.$$

Hence, the estimated computing capacity in 2025 is

$$C_{2025} = 40 \times 10^{12} \times 2^{10}.$$

Thus,

$$C_{2025} = 40 \times 10^{12} \times 1024 = 4.096 \times 10^{16}$$

instructions per second.

Therefore,

$$C_{2025} \approx 4.096 \times 10^{16} \text{ instructions/second}$$

under the assumptions of the exercise.

5. Execution time for 2^{100} instructions in 2025

The algorithm performs

$$2^{100} \approx 1.2677 \times 10^{30}$$

instructions.

Using the estimated 2025 computing capacity:

$$T_{2025} = \frac{2^{100}}{4.096 \times 10^{16}}.$$

Therefore,

$$T_{2025} \approx 3.095 \times 10^{13} \text{ seconds.}$$

Converting into years:

$$T_{2025} = \frac{3.095 \times 10^{13}}{365.25 \times 24 \times 3600}.$$

Hence,

$$T_{2025} \approx 9.81 \times 10^5 \text{ years}$$

or approximately

$$981,000 \text{ years}.$$

Thus, even after twenty years of computing-capacity improvement according to the assumed Moore's law model, the computation involving 2^{100} instructions remains completely impractical.

6. Why hardware improvement cannot compensate indefinitely

Suppose computing capacity is multiplied by a constant factor k . The execution time becomes

$$T'(n) = \frac{f(n)}{kC}.$$

Therefore, increasing computing capacity by a fixed factor just divides the execution time by that same factor.

For an exponential algorithm,

$$f(n) = 2^n,$$

the number of instructions doubles whenever n increases by just one:

$$2^{n+1} = 2 \cdot 2^n.$$

Consequently, increasing the input size produces an extremely rapid increase in execution time.

For example,

$$2^{100} \approx 1.27 \times 10^{30},$$

while

$$2^{200} \approx 1.61 \times 10^{60}.$$

The latter is approximately

$$2^{100}$$

times larger than the former.

Thus, hardware improvements alone cannot overcome the rapid growth associated with exponential algorithms.

7. Discussion: n^4 versus 2^n

Consider

$$f_A(n) = n^4$$

and

$$f_B(n) = 2^n.$$

For small values of n , the two functions can have relatively close values. For example,

$$6^4 = 1,296 \text{ while } 2^6 = 64.$$

$$\text{and } 10^4 = 10,000 \text{ while } 2^{10} = 1,024.$$

However, at $n = 20$:

$20^4 = 160,000$,

whereas

$2^{20} = 1,048,576$.

At $n = 50$:

$50^4 = 6,250,000$,

whereas

$2^{50} \approx 1.126 \times 10^{15}$.

The difference becomes enormous as n increases.

Therefore,

$$n^4 < 2^n \quad \text{for sufficiently large } n$$

and the polynomial-time algorithm becomes much more suitable for large input sizes.

Conclusion

The exercise illustrates a fundamental principle of algorithmic complexity: increasing computing capacity can reduce execution time, but it does not change the growth rate of an algorithm. An exponential algorithm such as 2^n can remain impractical even on extremely powerful computers, whereas polynomial algorithms such as n , $n \log n$, n^2 , n^3 , and n^4 generally scale much better as the input size increases.

Moore's law can provide substantial improvements in hardware performance, but these improvements are insufficient to overcome the exponential growth associated with computationally expensive algorithms.